



Customer Reply Activity

Technical Architecture and Operations Guide

A clear reference for implementation, support, and controlled maintenance.

This guide describes how Customer Reply Activity works with the standard Odoo mail and activity architecture. It is intended for Odoo administrators, implementation partners, developers, support engineers, and technical owners who need to understand processing order, data structures, reliability controls, security, and operational checks.

DOCUMENT Technical and Operations Guide	MODULE mail_customer_reply_activity
FORMAT A4 documentation set	SUPPORT full.stack.odoo@gmail.com

DOCUMENT MAP

Contents

Use this list to move directly to the section needed for configuration, daily work, support, or validation.

Section	Page
Technical Purpose and Scope	3
Standard Incoming Mail Context	3
Candidate Selection	4
Processing Sequence	5
Configuration Data Model	5
Activity Data Model	6
Responsible User Resolution	7
Deadline Calculation	7
Merge and Notification Lifecycle	7
Sender and Message Filtering	8
Reliability and Transaction Safety	9
Security and Access Control	10
Operational Monitoring	10
Deployment, Upgrade, and Recovery	11
Performance Considerations	11
Extension Boundaries	12
Technical Acceptance Checklist	12
Support Information	13

SECTION Technical Purpose and Scope

Customer Reply Activity closes the operational gap between receiving an external reply and assigning responsibility for the response. The module does not replace the mail gateway, message router, chatter, or standard activity system. It extends the normal incoming mail flow after Odoo has identified the target business record.

The central technical principles are:

- standard routing remains the source of truth for the email and target record;
- the accepted reply is stored in chatter before activity work begins;
- configuration is model-based instead of hard-coded for CRM or another application;
- assignment comes from the record's existing user field, with an optional fallback;
- filtering is applied to the activity decision, not to storage of the accepted email;
- creation and merge notifications follow the standard activity lifecycle without duplicate calls;
- savepoints and database locks protect the mail route and concurrent activity updates.

The module technical name is `mail_customer_reply_activity`. It depends on the standard `mail` add-on and adds no external Python package requirement.

SECTION Standard Incoming Mail Context

Odoo parses an incoming message through `mail.thread`. It reads identifiers and reply headers, finds a related parent message when possible, determines a route, and stores the accepted email as a `mail.message` on the target record.

Important message information includes:

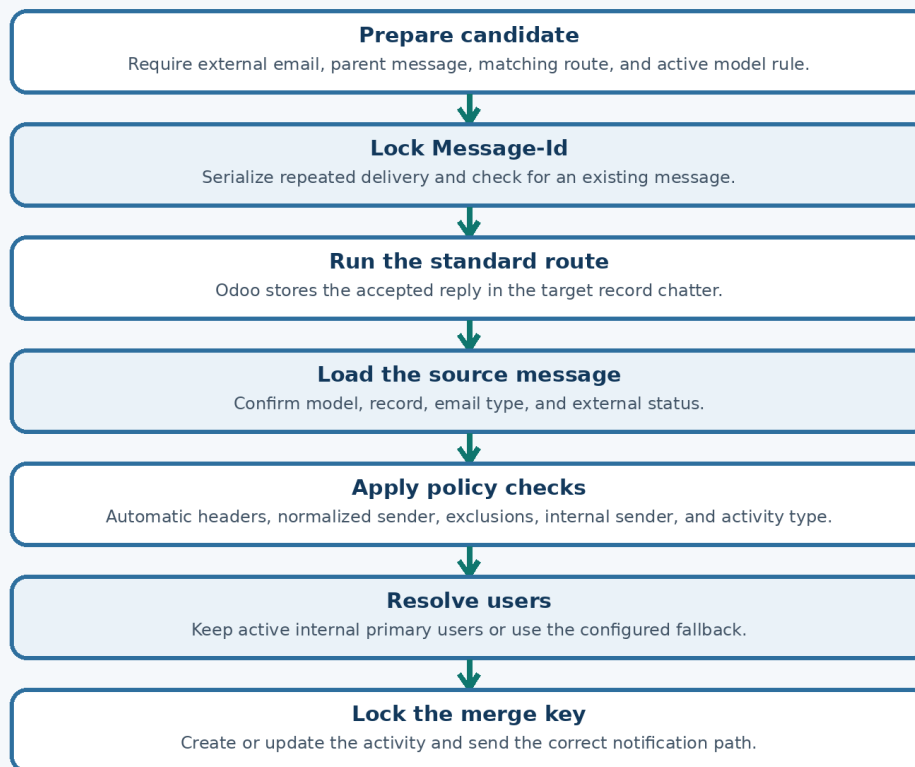
- `Message-Id`, which identifies the incoming message;
- `In-Reply-To` and `References`, which connect the incoming email to an earlier message;
- `parent_id`, which points to the resolved parent `mail.message`;
- the routed model and record ID;
- sender, subject, message type, internal-message status, MIME type, and mail headers.

A reply is a candidate only when the parent and final route agree on the same model and record. This avoids treating an unrelated alias email as a reply to an existing conversation.

Why the route stage is used

The route-processing stage provides both the resolved relationship and the final destination selected by Odoo. Earlier stages may not yet know the correct record. Later stages may no longer provide the original parsed headers needed for automatic-message checks.

The extension prepares candidate information before standard route completion, calls the standard route, and processes candidates afterward. This order keeps chatter as the primary communication record.

Processing order and failure isolation*Technical processing sequence around the standard mail route***SECTION Candidate Selection**

Candidate preparation is intentionally strict. A message enters the Customer Reply Activity decision only when all initial conditions are true:

- message type is email;
- the parsed message is not marked internal;
- a valid parent message exists;
- the parent message has a business model and record ID;
- a final route returns to that same model and record;
- an active rule exists for the routed model.

The module groups available rules by model. The database allows only one active rule for each model, so one route cannot be processed by several competing active configurations.

New alias messages

A new email sent to an alias without a valid reply relationship follows standard Odoo behavior. It may create or update a record according to the alias, but it is not processed as a Customer Reply Activity candidate.

Duplicate Message-Id behavior

When a candidate exists, a transaction-level lock is taken for the incoming Message-Id. The module checks whether a `mail.message` with that identifier already exists. A repeated copy of the same message is not routed and scheduled again.

This control protects against repeated delivery of one email. A genuine second reply normally has a different Message-Id and remains eligible.

SECTION Processing Sequence

After the standard route stores the message, each candidate is processed in an isolated savepoint. The processing sequence is:

1. Confirm that the rule is still active and the routed model still exists.
2. Confirm that the target record exists and supports activity scheduling.
3. Find the new routed `mail.message` by Message-Id, model, record, and email message type.
4. Stop if the stored message is internal.
5. Apply automatic-message detection when the rule enables it.
6. Normalize and validate the sender address.
7. Apply configurable sender exclusions.
8. Check whether the sender belongs to an internal user.
9. Confirm that the selected activity type is active and compatible with the model.
10. Resolve primary users and, when needed, the fallback user.
11. Create or merge one activity for each eligible responsible user.

The accepted chatter message is not removed when one of these activity checks stops processing.

SECTION Configuration Data Model

The central model is `mail.customer.reply.activity.rule`.

Field	Technical role	Main validation
<code>sequence</code>	Orders rules in the administration list.	No effect on duplicate model protection.
<code>active</code>	Enables or archives the rule.	Only one active rule is allowed per model.
<code>model_id</code>	Links the rule to <code>ir.model</code> .	Model must support mail thread and activities and must not be transient.
<code>name</code>	Stored related model name used as display name.	Follows the selected model.
<code>model</code>	Stored technical model string used for fast lookup.	Follows the selected model and is indexed.
<code>responsible_field_id</code>	Links to <code>ir.model.fields</code> .	Field must belong to the selected model, be Many2one or Many2many, and relate to <code>res.users</code> .
<code>fallback_user_id</code>	Stores the optional second-level owner.	Must be an active internal user.
<code>activity_type_id</code>	Selects the activity type.	Type must be active and generic or linked to the rule model.
<code>deadline_count</code>	Stores the reaction amount.	Database check prevents negative values.
<code>deadline_unit</code>	Stores days, weeks, or months.	Required selection.
<code>merge_replies</code>	Selects update or separate-activity behavior.	Boolean, enabled by default.
<code>ignore_automatic_messages</code>	Enables header-based automatic-message filtering.	Boolean, enabled by default.

Field	Technical role	Main validation
<code>excluded_addresses</code>	Stores sender patterns.	Parsed at runtime using whitespace, comma, and semicolon separators.

The active-model uniqueness rule is implemented at database level. It protects the configuration even when records are created through imports or external calls instead of the form view.

View-level domains

The form limits choices before server validation:

- Model shows only supported non-transient mail and activity models.
- Responsible Field shows only fields on the selected model that point to Users and use Many2one or Many2many.
- Fallback Responsible shows active non-shared users.
- Activity Type shows active generic types and active types for the selected model.

Server constraints repeat the important checks. This means unsupported values cannot be accepted only because they were supplied outside the normal form.

SECTION Activity Data Model

The module extends `mail.activity` with four fields.

Field	Purpose
<code>customer_reply_rule_id</code>	Identifies the rule that created or controls the activity.
<code>customer_reply_count</code>	Stores how many external replies are represented by this activity.
<code>customer_reply_last_message_id</code>	Links to the latest related chatter message.
<code>customer_reply_last_received_at</code>	Stores the latest processing timestamp.

The rule and latest-message links use `ondelete="set null"`. Historical activities therefore remain valid if a related optional reference disappears.

A partial database index covers active customer-reply activities by rule, model, record, and user. This index supports the merge lookup without adding the same cost to unrelated activities.

Activity note

The note is rebuilt for the latest reply and contains:

- a Customer reply received heading;
- sender text from the latest message;
- latest subject or a No subject fallback;
- received timestamp;
- reply count.

The complete conversation is intentionally not duplicated in the activity note. The related record chatter remains the place to read message bodies and earlier context.

SECTION Responsible User Resolution

The configured field is read dynamically from the routed business record. Runtime checks confirm that the field still exists and still points to `res.users`.

For a Many2one field, zero or one primary user is returned. For a Many2many field, all linked users are read. The module keeps only active internal users and removes duplicate user IDs.

Fallback processing starts only when no eligible primary user remains. The fallback is checked again at runtime, even though configuration constraints already require an active internal user.

The final result can be:

- one activity for one eligible primary user;
- several activities for several eligible primary users;
- one activity for the fallback;
- no activity when no eligible user exists.

The last case is logged as a warning. No arbitrary administrator is assigned.

Assignment and merge key

The responsible user is part of the merge key. This creates predictable separation between owners. When the record owner changes, a later reply can create a new activity for the new owner while the previous user's activity remains unchanged.

SECTION Deadline Calculation

The deadline uses `fields.Date.context_today()` with the responsible user's timezone. A `relativedelta` adds the configured number of days, weeks, or months.

Technical consequences:

- the result is a calendar date because standard activity deadlines are date fields;
- zero means the current date in the responsible user's timezone;
- two users in very different timezones can receive different base dates around midnight;
- month calculations follow standard calendar behavior for shorter months;
- merged replies do not recalculate the deadline;
- a new activity after completion receives a new calculation.

The reaction amount is an integer and cannot be negative.

SECTION Merge and Notification Lifecycle

Merge lookup

When merge is enabled, a transaction-level advisory lock protects the key formed from:

- rule ID;
- record model;
- record ID;
- user ID.

The module then searches the oldest active matching activity. Completed activities are inactive in the standard activity lifecycle and are not included.

New activity path

When no matching activity exists, `activity_schedule()` creates a standard activity. The context enables the standard quick activity update flow. Standard activity creation sends the assignment notification through `action_notify()`.

The module does not call `action_notify()` a second time after creation. This avoids duplicate notification work for the first reply.

Merged activity path

When a matching activity exists, the module writes the new activity type, summary, note, count, latest message, and latest timestamp. Standard `write()` does not notify an unchanged responsible user. The module therefore calls `action_notify()` once after the merge update.

This division gives:

- first reply: one standard creation notification;
- each merged reply: one explicit reminder;
- separate mode: one standard creation notification for each new activity.

Notification delivery still depends on the user's standard preferences and outgoing mail configuration. Failure to deliver one notification is isolated in a savepoint and logged; the activity update remains the main business result.

SECTION Sender and Message Filtering

Sender normalization

The sender is taken from the routed `mail.message` when available and normalized through standard Odoo mail utilities. An invalid or missing normalized address stops activity creation and produces a warning.

Configured exclusions

Exclusion patterns are split on whitespace, commas, and semicolons. Each pattern is lowercased. Matching uses wildcard-aware, case-sensitive comparison on already normalized lowercase values.

Pattern branches are:

- pattern begins with `@`: compare the domain with the remaining pattern;
- pattern contains no `@`: compare the domain with the full pattern;
- pattern contains `@`: compare the complete normalized address.

The wildcard syntax follows standard filename-style matching. Administrators should use narrow patterns and test allowed senders after changes.

Internal sender checks

The first check uses the chatter message author. If that contact is linked to any internal user, the sender is internal. The second check searches internal user partner emails by normalized address with inactive users included.

Including inactive user addresses helps prevent an old employee account from being treated as an external customer.

Automatic-message checks

When enabled, the classifier evaluates:

- `Auto-Submitted`, excluding empty and `no` values;

- Precedence tokens such as `auto-reply`, `auto_reply`, `bulk`, `junk`, and `list`;
- common autoreply and autoresponder headers;
- Exchange inbox-rule and mail-loop headers;
- `List-Id` and `List-Unsubscribe`;
- an empty `Return-Path` value;
- MIME content type `multipart/report`.

`X-Auto-Response-Suppress` alone is not a classifier. It describes which automatic responses should be suppressed and does not prove that the incoming message was created automatically.

Filtering happens after standard routing. A message can therefore be stored in chatter while no Customer Reply activity is scheduled.

SECTION Reliability and Transaction Safety

Savepoint boundaries

Candidate preparation, duplicate locking, candidate processing, individual user scheduling, and merged-reply notification use database savepoints. A failure in the added activity workflow is logged without intentionally rolling back the already accepted standard chatter message.

Message lock

A signed 64-bit advisory lock key is derived from the database name, module namespace, operation namespace, and `Message-Id`. The transaction lock serializes workers that receive the same message at the same time.

Activity lock

A second advisory lock serializes the merge key for one rule, record, and user. Without this lock, two mail workers could both search before either creates an activity. The lock makes the search-and-create or search-and-update sequence predictable.

Database-specific behavior

The concurrency control uses PostgreSQL transaction advisory locks, which match the standard Odoo database platform. Locks are released automatically when the transaction finishes.

Duplicate and concurrency outcomes

Situation	Expected result
Same Message-Id delivered twice	The duplicate is not routed and does not schedule a second activity.
Two different replies arrive together with merge enabled	Processing is serialized for the activity key, producing one open activity with the correct accumulated count.
Two different replies arrive together with merge disabled	Each reply creates its own activity.
Activity notification fails after merge	Activity update remains; the notification failure is logged.
Candidate-specific processing fails	Standard accepted chatter message remains available.

SECTION Security and Access Control

The rule model grants read, create, write, and delete access only to the system administration group. The Settings and Technical Email menu entries use the same administration group.

Daily users interact with standard activities and business records. Their visibility follows normal Odoo access rules for those records and activity features.

Important security points:

- fallback selection is limited to active internal users;
- primary assignment filters out portal, public, and inactive users;
- rule creation uses server validation, not only interface domains;
- activity processing uses elevated access so mail workers can schedule the required activity after a valid route;
- the module does not request or store mailbox passwords;
- sender exclusions are configuration data and should be changed only by trusted administrators.

Multi-company scope

The rule model has no company field. One active rule applies to its model throughout the database. Record ownership can still differ by company, but the reaction period, merge policy, filtering option, and sender patterns are shared for that model.

SECTION Operational Monitoring

The module does not introduce a separate monitoring dashboard. Administrators use standard Odoo tools and logs.

Useful administration areas

- **Customer Reply Activities:** review active and archived rules.
- **Activity Types:** confirm that the selected types remain active and compatible.
- **Technical > Email > Emails:** inspect outgoing activity notification records.
- **Incoming Mail Servers:** confirm reply retrieval and processing.
- **Business record chatter:** confirm the route and inspect the source message.
- **Activities:** review ownership, deadline, count, and completion.
- **Server logs:** identify invalid senders, missing responsible users, invalid activity types, or processing exceptions.

Log events worth monitoring

Event	Meaning	Administrator response
Failed to prepare candidates	An exception occurred before standard route completion.	Review the stack trace and current rule data; confirm standard routing still completed.
Failed to lock processing	Duplicate-protection lock or lookup failed.	Review database health and the stack trace.
Duplicated customer reply ignored	The same Message-Id already exists.	Usually informational; investigate repeated delivery only if frequent.

Event	Meaning	Administrator response
Sender address is invalid	The routed source message has no valid normalized sender.	Inspect the original email headers and sender format.
Rule has an invalid activity type	Activity type was changed outside the expected validation path or is no longer compatible.	Correct the rule and activity type.
Rule has no responsible user	Primary assignment and fallback both failed.	Correct record ownership or configure a fallback.
Failed to notify user	Activity exists or was updated, but notification work failed.	Inspect outgoing mail and user preferences.
Failed to schedule activity	A user-specific scheduling exception occurred.	Review record access, activity data, and the complete stack trace.

SECTION Deployment, Upgrade, and Recovery

Deployment sequence

1. Back up the database and add-on files according to the normal release process.
2. Place the module directory in the configured add-ons path.
3. Restart the Odoo service.
4. Update the Apps list and install or upgrade the module.
5. Open every active rule and review its linked model, responsible field, fallback, and activity type.
6. Run the first-reply, merged-reply, completed-activity, fallback, exclusion, automatic-message, and notification checks.
7. Monitor server and outgoing mail logs during the first business cycle.

Safe configuration recovery

If a rule must stop immediately, archive it. This is safer than deleting history or modifying database rows directly. The accepted email route remains standard, and existing activities remain available.

If an activity type must be retired, change every rule that uses it before deletion. If a responsible field is removed by another customization, update the rule to a valid user field and run a new reply test.

Uninstallation considerations

Before uninstalling, export or review open Customer Reply activities according to the business retention policy. Uninstallation should follow the normal add-on process and a verified database backup. Chatter messages are standard mail records and are separate from the rule configuration.

SECTION Performance Considerations

The module performs its main work only for incoming emails that have a parent message and a matching routed record. Rule lookup uses the stored technical model field. Merge lookup uses a partial index limited to active customer-reply activities.

Primary runtime work includes:

- one active-rule search for candidate models;

- one source-message lookup after routing;
- sender and header checks in memory;
- one user-resolution step;
- one indexed activity lookup per user when merge is enabled;
- one activity create or update per eligible user.

Many2many responsible fields increase activity work in direct proportion to the number of eligible users. Administrators should use team fields intentionally and avoid assigning very large user groups when one operational owner is more appropriate.

SECTION **Extension Boundaries**

The base module is designed as a focused response-control layer. Common extensions can build on the rule, activity link, reply count, and latest-message fields.

Possible separate extensions include:

- manager escalation after a defined period;
- different reaction targets for priority customers;
- team routing based on recipient or alias;
- Helpdesk integration;
- response-time reporting;
- external collaboration-channel notifications;
- business calendars and hour-level deadlines;
- company-specific rules;
- record-domain conditions.

These features should remain separate from the base reply-to-activity promise so the standard path stays easy to configure and support.

SECTION **Technical Acceptance Checklist**

- Module installs and upgrades through the normal registry process.
- Rule model, views, menus, access rights, translations, and activity type load.
- Supported models and user fields are correctly limited in the interface.
- Database constraints reject duplicate active rules and negative reaction values.
- Activity-type consistency is protected.
- A reply with valid parent headers reaches the exact original record.
- Standard chatter storage completes before activity processing.
- External eligible sender creates an activity.
- Internal, excluded, and confirmed automatic senders do not create activities.
- Many2one and Many2many assignment return only active internal users.
- Fallback is used only when primary assignment is empty or ineligible.
- Merge count, latest message, timestamp, note, and original deadline are correct.
- Completed activity is not reopened.
- First creation and each merge produce one notification path.
- Repeated Message-Id handling and concurrent merge logic remain protected.

- Missing assignment or activity-specific failure does not remove the accepted chatter message.

SECTION Support Information

Technical name: `mail_customer_reply_activity`

Dependency: `mail`

License: LGPL-3

Support contact: full.stack.odoo@gmail.com

For technical investigation, provide the rule values, routed model and record ID, incoming Message-Id, parent message reference, sender address, responsible field value, resulting activity state, and relevant server log section. Remove passwords, tokens, and private message content that is not required for diagnosis.